

Building Temporary Isolated Workspace in Real-Time Collaborative Programming Environment

Jinfeng Jiang, Yuxiang Xie, Bicheng Fang, Mingjie Wang, and Hongfei Fan*

School of Software Engineering, Tongji University

Shanghai, China

{2231513, 1951708, 2052329, 1851055, fanhongfei}@tongji.edu.cn

Abstract—Real-time collaborative programming supports a team of programmers to concurrently view and edit the same set of source code at the same time, which is beneficial in meeting particular collaboration needs. However, during the collaboration process, programmers are not able to compile and debug the source code with syntactic errors as it is being continuously edited by other collaborators. To address this challenge, we propose a novel approach named Reversion of Error-free Code with Workspace Isolation (RECON) and contribute supporting techniques with prototype implementation. In this approach, the system continuously monitors the files being collaboratively edited, detects source code without syntactic error, and maintains additional error-free source code copies. Whenever a programmer attempts to compile and debug the code, the system creates a temporary isolated workspace and replaces the source code files with the latest error-free copies. The proposed approach and solution have been implemented in a prototype system named CoIDEA, which has indicated the feasibility of the scheme and techniques.

Index Terms—Real-time collaborative programming, programming environments, error-free source code copy, workspace isolation

I. INTRODUCTION

Real-time collaborative programming is an emerging approach and technique, which supports a team of programmers to work on software projects in a closely-coupled fashion. During real-time collaboration, each programmer's editing operation is automatically transmitted to the server and instantly propagated and applied to collaborators' source code copies. Real-time collaborative programming supports instant awareness and feedback among programmers, which may help increase the efficiency and quality of teamwork [1]–[3].

Despite its benefits, this novel technique also introduces a set of challenges and issues. One major problem is that during the real-time collaboration session, an individual programmer is not able to test and debug the code from time to time (which is commonly practised in software development), because the active source code copy is being continuously changed by the collaborators, and it is very difficult to reach a point of time where all source code pieces are free of syntactic errors and thus compilable. To be concrete, a detailed illustration of this issue is presented in Fig. 1 below, where one programmer has completed *fun1* and attempts to

compile and debug the code, while *fun2* is still being edited by another programmer, and thus the code is not compilable.

```
class Example {  
    String fun1() {return "Hello World";}  
    String fun2() {return "Still editing..."  
}
```

Fig. 1. An example of uncompileable source code during real-time collaborative programming.

Under such circumstances, collaborating programmers have to spend additional coordination effort to address the need of compilation, testing and debugging. For example, the programmers may collectively decide a time for such activities, where each individual programmer needs to keep the source code in a compilable state (i.e. other planned work has to be postponed). It is obvious that such communication and coordination will lead to significantly negative impact on the efficiency of teamwork.

To address such challenge, we propose a novel approach named *Reversion of Error-free Code with Workspace Isolation* (RECON) in this study. While each collaborating programmer views and modifies the latest source code as usual, the RECON-embedded real-time collaborative programming environment continuously analyzes the source code and maintains additional source code copies (versions) without syntactic error. Whenever a programmer attempts to compile and debug the code while other programmers are simultaneously making changes, the latest working version of the source code could be temporarily reverted to the error-free copy, thus providing a clean and isolated private workspace for supporting compilation and debugging.

The major contributions of this study are briefly summarized as follows. Firstly, we have proposed a novel approach of building temporary isolated workspace for supporting compilation and debugging activities. Secondly, for supporting the proposed approach, we have proposed a file-based version management mechanism and a systematic scheme for reverting error-free source code copies back into the isolated workspace. Thirdly, we have implemented a prototype system that validates the feasibility of the approach and techniques.

The rest of this paper is organized as follows. In Section II, we present a brief review on real-time collaborative programming environments and techniques, and analyze the limitations and challenges identified. In Section III, we

* Corresponding Author: Hongfei Fan

introduce the proposed RECON approach. In Section IV, we present the major scheme and supporting techniques under the RECON approach. In Section V, we present the implementation of a prototype system, and report the results of experimental evaluations. Finally, we conclude this study by providing a summary of our contributions, and discussing potential directions for future research in Section VI.

II. RELATED WORK

Real-time collaborative programming is an emerging approach that enables multiple programmers to concurrently access and edit any part of a shared source code copy at the same time. With rising interests in this promising field, a number of research prototypes have been developed in the recent years, such as *Saros* [4], *CoEclipse* [5], *CodeSync* [6], *CodeR* [7], *CoVSCode* [8], *COLLECE* [9], and *CoIDEA* [10]. In addition, there also exists several real-time collaborative programming tools (plugins) officially released with widely-used IDEs, such as *Visual Studio Live Share* [11] and *Code With Me* [12].

Technically, real-time collaborative programming environments rely on a set of key technologies that are generic for real-time collaborative editing systems. To achieve high local responsiveness, the shared documents (e.g. shared source code files in the domain of real-time collaborative programming) are commonly replicated at all collaborating sites, so that local editing operations can take effect immediately and then propagated to remote sites. Consequently, it is particularly critical to ensure the consistency of the shared document copies at different sites. Operation transformation (OT) [13]–[16] is a mature and sophisticated technique for consistency maintenance, which has been widely applied in a number of real-time collaborative editing systems [8], [17]–[20]. In addition, there also exists alternative consistency maintenance techniques with diverse features, such as commutative replicated data type (CRDT) [21]–[23] and address space transformation (AST) [24]–[26].

On the other hand, for supporting real-time collaborative programming in various aspects, the community has contributed a number of detailed techniques, such as the integration of real-time and non-real-time collaboration [18], [27], operation merging for optimized performance [28], and semantic conflict prevention [29], [30], which have significantly improved the usability of real-time collaborative programming from various aspects.

Unfortunately, as an emerging technique, real-time collaborative programming has not yet been widely used in practice due to a number of open problems. One major issue is the lack of support on programmers' compilation and debugging activities during the collaboration process, which has been illustrated in Section I and is the focus of this study. As an attempt to address such requirement, Max et al. [31] proposed an approach that facilitates error-free debugging of collaboratively edited code with a prototype implementation in a web-based real-time collaborative programming platform named *Collabode*. Given a piece of source code, it maintains a union version (storing all collaborators' real-time work) and

a disk version (a temporary version without syntactic error). Technically, for each series of operations, the system extracts the largest subset of the operations that can make up an error-free version and applies them to the disk version. Whenever the user compiles the code, the disk version would be used. This approach provides a potential solution for supporting debugging sessions during real-time collaborative programming process; however, the error-free version determination incurs great computational cost in the presence of a large number of operations, and it forces the users to compile the disk version without flexibility. In this study, we propose an efficient and flexible approach to satisfy the compilation and debugging requirements in real-time collaborative programming.

III. APPROACH AND CONCEPTS

A. The RECON Approach

To address the need of compilation and debugging during real-time collaborative programming, we propose a *Reversion of Error-free Code with Workspace Isolation* (RECON) approach, which is briefly sketched as follows with an illustration in Fig. 2.

- 1) During the collaboration process, each programmer views and edits the latest working version of the shared source code in their workspace as before, which contains all real-time edits from the team.
- 2) Simultaneously, each client system automatically maintains and updates an additional error-free source code copy, which contains the latest changes made by the team without any syntactic error (thus compilable).
- 3) Whenever a programmer attempts to compile and debug the code, an isolated workspace will be generated with its contents temporarily reverted to the latest error-free copy.

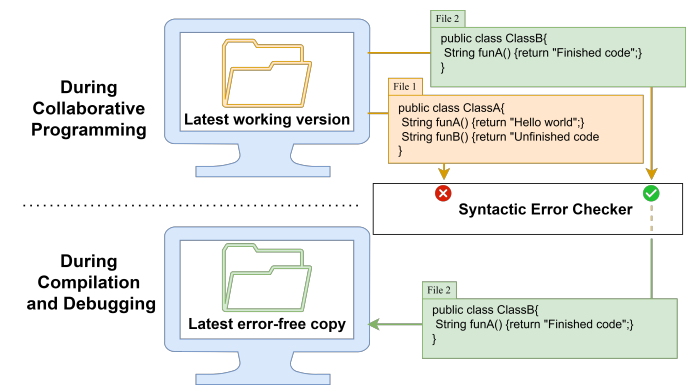


Fig. 2. A general sketch of the RECON approach.

B. Core Concepts

For further elaboration of the proposed approach, we introduce several core concepts as follows:

- 1) The programming environment works in either of the following two modes:

- *Collaboration Mode*: the regular mode in which programmers conduct real-time collaborative programming, where all local and remote operations are instantly applied and merged.
 - *Silent Mode*: the special mode in which the system has replaced the source code with the latest error-free copy, and the local workspace is temporally isolated from collaboration.
- 2) The programming environment maintains two source code repositories in the local storage:
 - *Base Repository*: the regular and active repository, in which the contents can be displayed and manipulated in the IDE's workspace in both working modes.
 - *Shadow Repository*: the additional repository that stores the latest error-free copy corresponding to the latest working version in the collaboration mode.
 - 3) The programming environment maintains two versions of source code:
 - *Base Version*: the version of source code stored within the base repository in the collaboration mode (i.e. the latest working version).
 - *Shadow Version*: the version of source code stored within the shadow repository in the collaboration mode (i.e. the latest error-free copy).

IV. SCHEME AND TECHNIQUES

Following the general approach, we have devised a detailed working scheme for supporting the RECON approach, as presented in Fig. 3.

Each programmer's client system contains two key components named *difference table* and *syntactic error checker*. The difference table is a data structure that documents the information of the files that differ between the base version and the shadow version, thus providing a clearer picture of the different files between two repositories; and the syntactic error checker is a component that identifies syntactic errors for a given source code file, serving as a cornerstone in the process of shadow version maintenance.

In the collaboration mode, each client system preserves the real-time source code transmission with the server and updates the shadow version with the assistance of the syntactic error checker.

After switching to the silent mode, the system creates an isolated workspace for the local programmer by swapping the different files between the two repositories by consulting the difference table, and simultaneously, maintains an additional server-to-client transmission for recording remote editing operations during the period of silent mode.

In the following subsections, we present detailed elaborations of the supporting techniques.

A. Difference Table Maintenance

In both working modes, the system monitors programmers' operations from local and remote sites. Specifically, the system manages to distill information such as operation type, operation position, and operation content. The operation information is then transformed and appended to the

difference table. Whenever a file in the shadow repository has been successfully updated, the corresponding entry will be removed from the table.

B. Shadow Version Update Mechanism

During real-time collaborative programming, programmers work in the collaboration mode where operations are applied directly to the files of the base repository. Concurrently, the syntactic error checker is invoked to check the altered file for syntactic errors. Should the result indicates no such error, the modified file is then committed to the shadow repository. In order to enhance the precision and comprehensiveness of identifying and updating the shadow version, the system employs four more distinct methods in addition to the previously mentioned one:

- 1) Update the version of the corresponding file in the shadow repository when a file has been created/deleted.
- 2) In the silent mode, the operations of other users will be applied to the shadow repository. These actions are also recorded, and upon exiting the silent mode and entering the collaboration mode, syntactic error detecting and shadow version updating are conducted on the relevant files based on the recorded actions.
- 3) Upon successful execution by the user, overwrite the files with differences according to the difference table into the shadow repository.
- 4) When the user manually stores a shadow version of a file, the corresponding file in the shadow repository is updated directly, regardless of the code's syntactic correctness.

C. Working Mode Switch Mechanism

When the user is about to perform compilation and debugging work on the code, the client's working mode switches from the collaboration mode to the silent mode via a list of well-defined steps as illustrated in Fig. 4:

Step 1: Pause the update process for shadow versions and the transmission of information from the current client to the server, to prevent the switch operation from being synchronized to other collaborators.

Step 2: Swap the files that differ between the two repositories according to the difference table.

Step 3: Redirect other programmers' collaborative operations to the corresponding files in the shadow repository to ensure timely receipt of collaborative effort from other collaborators during the period of silent mode.

Following the above steps, the files in the base repository are in the shadow version, and the files in the shadow repository are in the base version. Programmers shall then be presented with the error-free copy saved in the shadow repository, which they can subsequently compile and debug.

Upon completion of the debugging process, the programmer can choose to leave the silent mode and switch back to the collaboration mode, and the system will behave as follows:

Step 1: Swap the files that differ between the two repositories according to the difference table.

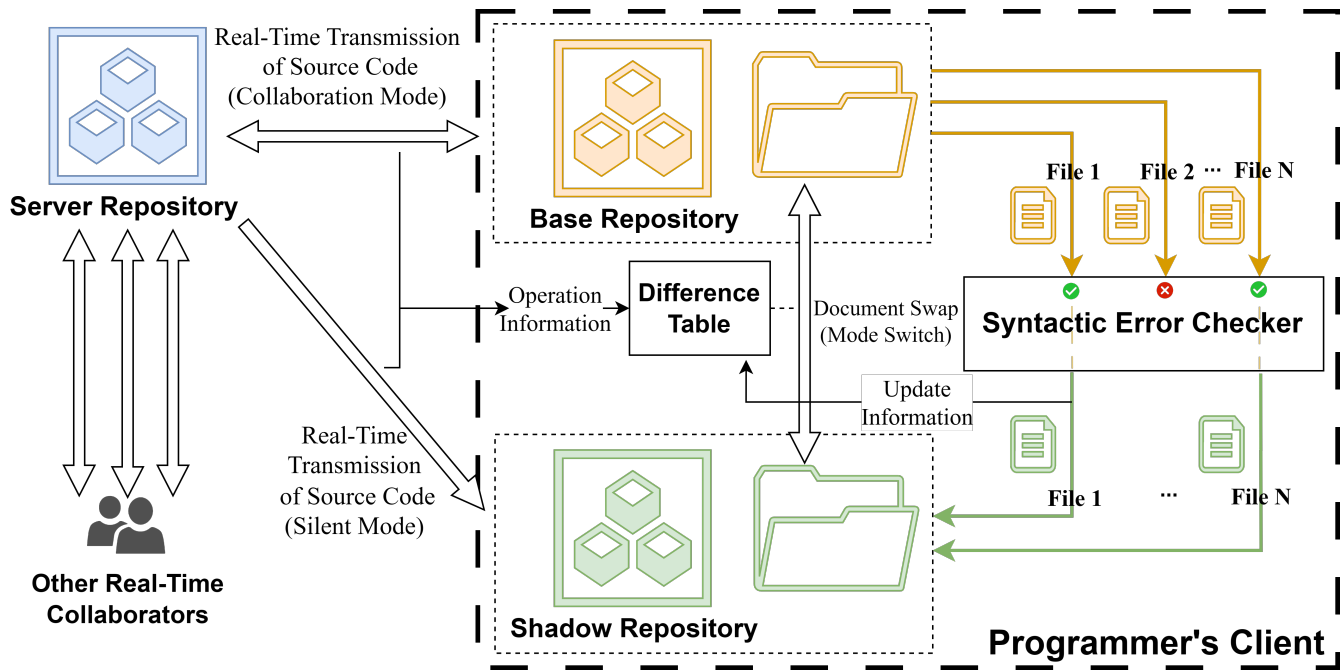


Fig. 3. The working scheme and architecture of RECON.

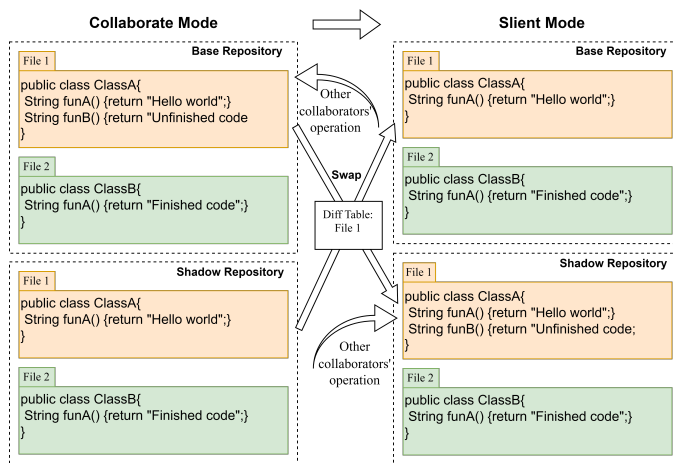


Fig. 4. Switching from collaboration mode to silent mode.

Step 2: Redirect the remote editing operations from other programmers (as received and accumulated during the period of silent mode) to the corresponding files in the base repository.

Step 3: Resume the transmission of editing operations from the current client to the server, and resume the shadow version update process as well.

The above steps look similar as the steps during the switch from the collaboration mode to the silent mode. It is worth mentioning that the files swapped between the two mode switch processes may not be identical. For instance, subsequent to entering the silent mode, files that were not previously present in the difference table may have been modified by other collaborators. Directly swapping the files based on the exchanged files that have undergone the switch

from collaboration mode to silent mode may lead to a range of adverse effects, such as a mismatch between the base repository and the server repository, or erroneous files in the shadow repository. Thus, both transitions should be dependent on the difference table.

V. IMPLEMENTATION AND EVALUATION

CoIDEA [10] is an IntelliJ IDEA plugin for supporting real-time collaborative programming, which was developed in our prior study, and has been continuously serving as a research vehicle for investigation and evaluation. In this study, we utilize *CoIDEA* as the foundation for implementing and evaluating the RECON approach, techniques, and solution.

Based on the implemented prototype system, we have designed and conducted a set of evaluations.

Fig. 5 presents the user interface of the real-time collaborative programming environment with detailed indications on RECON-related features. In the scenario shown in the upper part (i.e. in collaboration mode), two programmers (namely Alice and Bob) are working on the *User* class together. When Bob is still editing the code, Alice attempts to compile the code to test the results of her work, but at this time, the code is in an uncompileable state. By clicking the *Enter Silent Mode* button in the upper right corner, the code in the workspace is immediately transformed into the latest error-free copy in the shadow directory, which Alice can successfully compile and debug. Meanwhile, the icon of the button is changed to indicate the working mode switch.

After the mode transition, the user interface of Alice's client will be presented as the lower part of Fig. 5, where the content of the *User* class has been replaced as the error-free copy. On the other hand, the button in the upper right corner is changed from *Enter Silent Mode* to *Enter Collaboration*

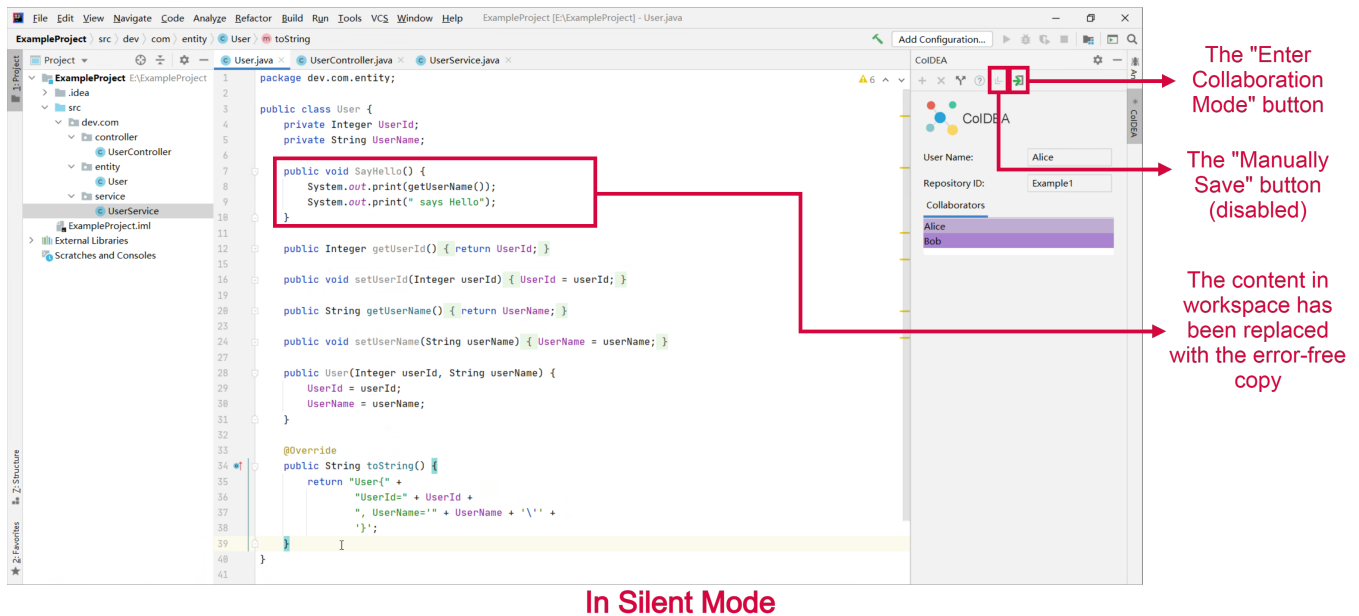
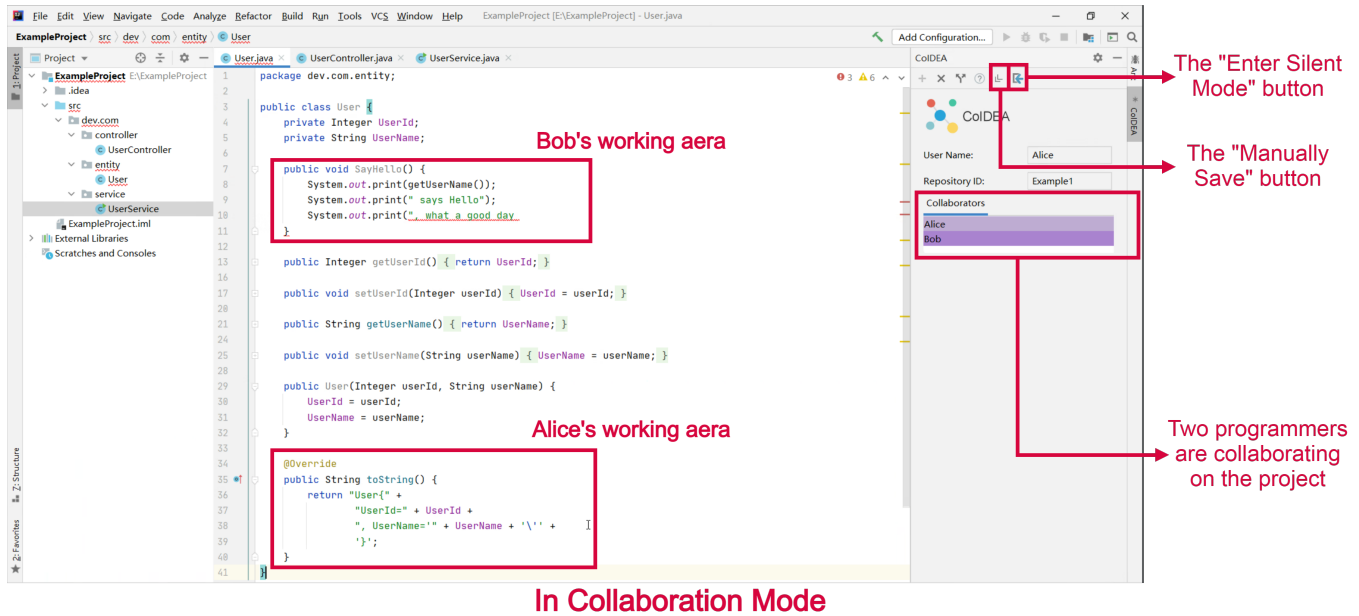


Fig. 5. The user interface of CoIDEA with RECON (in Alice's client).

Mode, which Alice can click after completing the debugging work to resume collaborative work with Bob.

VI. CONCLUSION AND FURTHER WORK

Real-time collaborative programming benefits the process of software development from various aspects. However, due to the nature of concurrent work, the source code copy is not in a compilable state during most of the time, and thus it is difficult for an individual programmer to perform testing and debugging tasks during the programming process. In this study, we propose a novel approach named *Error-free Code with Workspace Isolation* (RECON), which creates a temporary isolated workspace and replaces the source code copy with the latest error-free version for supporting compilation

and debugging purpose. We presented a general sketch of the proposed approach, defined core concepts, and devised a detailed working scheme with supporting techniques. A prototype system has been successfully implemented, and we have also conducted evaluations based on the prototype system, which have demonstrated the feasibility of the approach.

We are continuously working in the area of real-time collaborative programming, and have identified several issues for future exploration. Firstly, the current scheme triggers syntactic error detection for every user operation, which may result in unnecessary resource utilization, and it is desirable to devise potentially optimized mechanism. Secondly, the current scheme does not allow editing operations in the silent mode, and we plan to investigate the possibility of proper

relaxation on this constraint, so that the scheme would better fit programmers' traditional working habits during debugging session.

ACKNOWLEDGMENT

This study has been sponsored by the Natural Science Foundation of Shanghai (No. 21ZR1465100 and No. 23ZR1465300), the National Natural Science Foundation of China (No. 61702374 and No. 62172301), the Fundamental Research Funds for the Central Universities (No. 22120220658 and No. 22120210545), and the Shanghai Municipal Science and Technology Major Project (2021SHZDZX0100).

REFERENCES

- [1] J. T. Nosek, "The case for collaborative programming," *Communications of the ACM*, vol. 41, no. 3, pp. 105–108, 1998.
- [2] A. Y. Wang, A. Mittal, C. Brooks, and S. Oney, "How data scientists use computational notebooks for real-time collaboration," *Proceedings of the ACM on Human-Computer Interaction*, vol. 3, no. CSCW, pp. 1–30, 2019.
- [3] S. Sankaranarayanan, S. R. Kandimalla, C. A. Bogart, R. C. Murray, M. Hilton, M. F. Sakr, and C. P. Rosé, "Collaborative programming for work-relevant learning: Comparing programming practice with example-based reflection for student learning and transfer task performance," *IEEE Transactions on Learning Technologies*, vol. 15, no. 5, pp. 594–604, 2022.
- [4] S. Salinger, C. Oezbek, K. Beecher, and J. Schenk, "Saros: an eclipse plug-in for distributed party programming," in *Proceedings of the 2010 ICSE Workshop on Cooperative and Human Aspects of Software Engineering*, pp. 48–55, 2010.
- [5] H. Fan and C. Sun, "Achieving integrated consistency maintenance and awareness in real-time collaborative programming environments: The coeclipse approach," in *Proceedings of the 2012 IEEE 16th International Conference on Computer Supported Cooperative Work in Design (CSCWD)*, pp. 94–101, IEEE, 2012.
- [6] M. S. Feldman, "Codesync: A collaborative coding environment for novice web developers," *Wellesley College: Wellesley, MA, USA*, 2014.
- [7] A. Kurniawan, C. Soesanto, and J. E. C. Wijaya, "Coder: Real-time code editor application for collaborative programming," *Procedia Computer Science*, vol. 59, pp. 510–519, 2015.
- [8] H. Fan, K. Li, X. Li, T. Song, W. Zhang, Y. Shi, and B. Du, "Covscode: a novel real-time collaborative programming environment for lightweight ide," *Applied Sciences*, vol. 9, no. 21, p. 4642, 2019.
- [9] C. Lacave, M. A. García, A. I. Molina, S. Sánchez, M. A. Redondo, and M. Ortega, "Collece-2.0: A real-time collaborative programming system on eclipse," in *2019 International Symposium on Computers in Education (SIIE)*, pp. 1–6, IEEE, 2019.
- [10] Y. Ma, Z. Yang, B. Chiu, Y. Zhang, J. Jiang, B. Du, and H. Fan, "Supporting cross-platform real-time collaborative programming: Architecture, techniques, and prototype system," in *Collaborative Computing: Networking, Applications and Worksharing: 17th EAI International Conference, CollaborateCom 2021, Virtual Event, October 16-18, 2021, Proceedings, Part II* 17, pp. 124–143, Springer, 2021.
- [11] "Visual studio live share." <https://visualstudio.microsoft.com/services/live-share/>. Accessed: Apr. 19, 2023.
- [12] "Code with me." <https://www.jetbrains.com/code-with-me/>. Accessed: Apr. 19, 2023.
- [13] C. Sun, X. Jia, Y. Zhang, Y. Yang, and D. Chen, "Achieving convergence, causality preservation, and intention preservation in real-time cooperative editing systems," *ACM Transactions on Computer-Human Interaction (TOCHI)*, vol. 5, no. 1, pp. 63–108, 1998.
- [14] D. Sun and C. Sun, "Context-based operational transformation in distributed collaborative editing systems," *IEEE Transactions on Parallel and Distributed Systems*, vol. 20, no. 10, pp. 1454–1470, 2009.
- [15] Y. Xu and C. Sun, "Conditions and patterns for achieving convergence in ot-based co-editors," *IEEE Transactions on Parallel and Distributed Systems*, vol. 27, no. 3, pp. 695–709, 2015.
- [16] D. Sun, C. Sun, A. Ng, and W. Cai, "Real differences between ot and crdt in correctness and complexity for consistency maintenance in co-editors," *Proceedings of the ACM on Human-Computer Interaction*, vol. 4, no. CSCW1, pp. 1–30, 2020.
- [17] F. Liu, S. Xia, H. Shen, and C. Sun, "Comaya: incorporating advanced collaboration capabilities into 3d digital media design tools," in *Proceedings of the 2008 ACM conference on Computer supported cooperative work*, pp. 5–8, 2008.
- [18] H. Fan, C. Sun, and H. Shen, "Atcope: any-time collaborative programming environment for seamless integration of real-time and non-real-time teamwork in software development," in *Proceedings of the 2012 ACM International Conference on Supporting Group Work*, pp. 107–116, 2012.
- [19] B. Cho, A. Ng, and C. Sun, "Covim+ coemacs: a heterogeneous co-editing system as a potential solution to editor war," in *Cooperative Design, Visualization, and Engineering: 15th International Conference, CDVE 2018, Hangzhou, China, October 21–24, 2018, Proceedings 15*, pp. 64–68, Springer, 2018.
- [20] B. Cho, C. Sun, and A. Ng, "Issues and experiences in building heterogeneous co-editing systems," *Proceedings of the ACM on Human-Computer Interaction*, vol. 3, no. GROUP, pp. 1–28, 2019.
- [21] X. Lv, F. He, W. Cai, and Y. Cheng, "A string-wise crdt algorithm for smart and large-scale collaborative editing systems," *Advanced Engineering Informatics*, vol. 33, pp. 397–409, 2017.
- [22] X. Lv, F. He, W. Cai, and Y. Cheng, "Supporting selective undo of string-wise operations for collaborative editing systems," *Future Generation Computer Systems*, vol. 82, pp. 41–62, 2018.
- [23] W. Cai, F. He, and X. Lv, "Multi-core accelerated crdt for large-scale and dynamic collaboration," *The Journal of Supercomputing*, vol. 78, no. 8, pp. 10799–10828, 2022.
- [24] J. Zhang, T. Lu, H. Xia, B. Shao, and N. Gu, "Asts: A string-wise address space transformation algorithm for real-time collaborative editing," in *2017 IEEE 21st International Conference on Computer Supported Cooperative Work in Design (CSCWD)*, pp. 162–167, IEEE, 2017.
- [25] Z. Xu, C. Liu, S. Wang, P. Zhang, T. Lu, and N. Gu, "An ast-based consistency maintenance scheme for cross-chain digital assets," *CCF Transactions on Pervasive Computing and Interaction*, vol. 4, no. 2, pp. 142–157, 2022.
- [26] W. Cai, F. He, S. Yang, X. Lv, and Y. Cheng, "Self-compressing object sequence for consistency maintenance in co-editors," *Software: Practice and Experience*, vol. 52, no. 8, pp. 1802–1825, 2022.
- [27] Y. Ma, B. Qi, W. Xu, M. Wang, B. Du, and H. Fan, "Integrating real-time and non-real-time collaborative programming: Workflow, techniques, and prototypes," *Proceedings of the ACM on Human-Computer Interaction*, vol. 7, no. GROUP, pp. 1–19, 2023.
- [28] H. Zhou, Y. Ma, W. Xu, M. Wang, B. Du, and H. Fan, "Context-based operation merging in real-time collaborative programming environments," in *2022 IEEE 25th International Conference on Computer Supported Cooperative Work in Design (CSCWD)*, pp. 1426–1431, IEEE, 2022.
- [29] H. Fan, H. Zhu, Q. Liu, Y. Shi, and C. Sun, "A novel dal scheme with shared-locking for semantic conflict prevention in unconstrained real-time collaborative programming," *IEEE Access*, vol. 5, pp. 22566–22583, 2017.
- [30] W. Xu, Y. Zhang, B. Chiu, D. Chen, J. Jiang, B. Du, and H. Fan, "Hybrid semantic conflict prevention in real-time collaborative programming," in *Collaborative Computing: Networking, Applications and Worksharing: 17th EAI International Conference, CollaborateCom 2021, Virtual Event, October 16-18, 2021, Proceedings, Part II*, pp. 104–123, Springer, 2022.
- [31] M. Goldman, G. Little, and R. C. Miller, "Real-time collaborative coding in a web ide," in *Proceedings of the 24th annual ACM symposium on User interface software and technology*, pp. 155–164, 2011.